

# Reviewer Pack — Minimal Symplectic Volume of Quadrics and Resolution Proof W...

Entry 6386679d5e69 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 14:17:42 UTC

## Minimal Symplectic Volume of Quadrics and Resolution Proof Width Lower Bound

**Entry ID:** 6386679d5e69 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 14:17:42 UTC

### 1. Conjecture

**Field A** (mathematical branch): Symplectic Geometry **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

#### Statement:

For every  $d$ -dimensional Tseitin formula  $\varphi_G$ , the minimal symplectic volume ( $\text{MSV}(\varphi_G)$ ) of its associated quadric surface is linearly correlated with its resolution proof width  $w(\varphi_G)$ , such that  $\text{MSV}(\varphi_G) = \Omega(w(\varphi_G))$ .

#### Rationale (proposer's reasoning):

Symplectic geometry has not been widely applied in complexity theory, and minimal symplectic volume provides a geometric invariant that could potentially expose non-trivial structure related to proof complexity. The correlation with resolution proof width could reveal new insights into the intrinsic difficulty of Tseitin formulas.

**Taxonomy category:** SymplecticGeometry (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** c2749f60dfc728f3

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

The conjecture is supported if at least 80% of seeds ( $n=30$ ) show a correlation coefficient between minimal symplectic volume  $\text{MSV}(\varphi_G)$  and resolution proof width  $w(\varphi_G)$  greater than or equal to 0.8, with an average  $\text{MSV}(\varphi_G)/w(\varphi_G)$  ratio across all seeds greater than or equal to 1.

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 1.00       | SAFE             | SAFE             |
| NATURAL_PROOFS | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| KARP_LIPTON    | SAFE    | 1.00       | SAFE             | UNCERTAIN        |

## 4. Novelty audit

**Verdict:** ADJACENT\_OK against 4 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - "symplectic geometry" AND "resolution proof width" AND quadric surface" - "Tseitin formula" AND minimal symplectic volume" AND Boolean satisfiability" - "quadric surface" AND resolution proof complexity" AND symplectic geometry"

**Top relevant hits considered:** - [http://arxiv.org/abs/2105.06744v4] A Separator Theorem for Hypergraphs and a CSP-SAT Algorithm - [http://arxiv.org/abs/2203.01031v2] Residual categories of quadric surface bundles - [http://arxiv.org/abs/0908.3713v3] Walls in supersymmetric massive nonlinear sigma model on complex quadric surface - [http://arxiv.org/abs/1706.01358v4] Quadric surface bundles over surfaces and stable rationality

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

### 5.1 Generated Python source

```
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_d_regular_graph(d, n):
        if d * (n - 1) % 2 != 0:
            return None
        graph = {i: set() for i in range(n)}
        edges = []
        for i in range(n):
            for j in range(i + 1, n):
                if len(graph[i]) < d and len(graph[j]) < d:
                    graph[i].add(j)
                    graph[j].add(i)
                    edges.append((i, j))
        return graph

    def tseitin_formula(graph):
        literals = {node: f'x{node}' for node in range(len(graph))}
        clauses = []
        for node, neighbors in graph.items():
            clause = [f'~{literals[node]}']
            for neighbor in neighbors:
                clause.append(literals[neighbor])
            clauses.append(clause)
        for i in range(len(neighbors)):
            clause = [f'x{i}']
            for neighbor in neighbors[i]:
                clause.append(~literals[neighbor])
            clauses.append(clause)
```

```

        for j in range(i + 1, len(neighbors)):
            clauses.append([f'-{literals[neighbors[i]]}', f'-{literals[neighbors[j]]}', literals[neighbors[j]]])
    return clauses

def gaussian_elimination(matrix):
    n = len(matrix)
    m = len(matrix[0])
    for i in range(n):
        if matrix[i][i] == 0:
            for j in range(i + 1, n):
                if matrix[j][i] != 0:
                    matrix[i], matrix[j] = matrix[j], matrix[i]
                    break
            else:
                return None
        pivot = Fraction(matrix[i][i])
        for j in range(m):
            matrix[i][j] /= pivot
        for j in range(n):
            if j == i:
                continue
            factor = matrix[j][i]
            for k in range(m):
                matrix[j][k] -= factor * matrix[i][k]
    return matrix

def determinant(matrix):
    n = len(matrix)
    det = Fraction(1)
    for i in range(n):
        pivot = matrix[i][i]
        if pivot == 0:
            return 0
        det *= pivot
        for j in range(i + 1, n):
            factor = matrix[j][i] / pivot
            for k in range(m):
                matrix[j][k] -= factor * matrix[i][k]
    return det

def minimal_symplectic_volume(graph):
    n = len(graph)
    A = [[0] * (2 * n) for _ in range(2 * n)]
    for i in range(n):
        A[i][i] = 1
        A[n + i][n + i] = 1
        for j in graph[i]:
            A[i][n + j] = -1
            A[n + i][j] = -1
    det_A = determinant(A)
    if det_A == 0:
        return None
    det_A_inv = Fraction(1, det_A)
    return det_A_inv

def resolution_width(clauses):

```

```

    n = len(clauses)
    width = [len(c) for c in clauses]
    return max(width)

n_max = 40
instances_tested = 0
total_msv = Fraction(0)
total_w = 0

for _ in range(30):
    d = random.randint(2, 5)
    n = random.randint(5, n_max)
    graph = generate_d_regular_graph(d, n)
    if graph is None:
        continue
    clauses = tseitin_formula(graph)
    msv = minimal_symplectic_volume(graph)
    if msv is None:
        continue
    w = resolution_width(clauses)
    total_msv += msv * w
    total_w += w
    instances_tested += 1

if instances_tested == 0:
    return {
        "metric_name": "MSV/w",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

mean_msv_w = total_msv / instances_tested
correlation_coefficient = Fraction(total_msv * total_w, instances_tested * total_msv * total_w)
conjecture_holds = correlation_coefficient >= Fraction(8, 10) and mean_msv_w >= 1

return {
    "metric_name": "MSV/w",
    "metric_value": float(mean_msv_w),
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else "mapping_undefined"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

```

```

mean_msv_w = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_msv_w} std=0.0 support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_msv_w} std=0.0 support_fraction={support_fraction}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7c48833c.py", line 152, in <module>
    trial_result = run_trial(seed)
                    ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7c48833c.py", line 114, in run_trial
    clauses = tseitin_formula(graph)
               ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7c48833c.py", line 44, in tseitin_formu
    clauses.append([f'-{literals[neighbors[i]]}', f'-{literals[neighbors[j]]}', literals[node]])
                    ~~~~~^
TypeError: 'set' object is not subscriptable

```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test code crashed before producing data, which means it did not complete the required computations to verify the conjecture. | next: Re-run the test with proper error handling and ensure that it completes without crashing.

## 11. Audit log (LLM calls)

**Total LLM calls:** 9

| # | Phase           | Provider      | Model       | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|-------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | glm4:latest | 0         | 0   | 18614        |
| 2 | preregistration | ollama_remote | glm4:latest | 0         | 0   | 9910         |

| # | Phase    | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|----------|---------------|------------------|-----------|-----|--------------|
| 3 | novelty  | ollama_remote | glm4:latest      | 0         | 0   | 10445        |
| 4 | novelty  | ollama_remote | glm4:latest      | 0         | 0   | 10223        |
| 5 | test_gen | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 55253        |
| 6 | test_gen | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 8258         |
| 7 | test_gen | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 16063        |
| 8 | test_gen | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 15999        |
| 9 | judge    | ollama_remote | glm4:latest      | 0         | 0   | 15921        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 160687 ms total latency. Provider mix: {'ollama\_remote': 9}

(full prompt+response transcripts available in `research/audit/6386679d5e69.jsonl`)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/6386679d5e69.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/6386679d5e69.tar.gz` (if generated)